Strings & Factors

PSY 410: Data Science for Psychology

Dr. Sara Weston

2026-05-11

The Power Play

Something new in the team challenge

The top 2 teams on the scoreboard right now get a **one-time strategic choice:**

Option A — Boost: Add **+3 points** to your own team's total

Option B — Sabotage: Remove **2 points** from each of **two other teams** of your choice

Choices are final. First place picks first.

Strings: The basics

How many majors are in this dataset?

id	major
1	Psychology
2	psychology
3	Psych
4	PSYCHOLOGY
5	Biology
6	biology
7	Bio

How many majors are in this dataset?

R says **7**. You meant **2**.

The problem is inconsistent text — different cases, abbreviations, trailing spaces.

Today we learn to fix that.

What are strings?

Strings are text data — anything in quotes:

```
participant_name <- "Jane Doe"
diagnosis <- "Major Depressive Disorder"
feedback <- "The task was confusing"
```

The `stringr` package (part of tidyverse) gives you tools for working with strings. All `stringr` functions start with `str_`.

Creating and combining strings

```
first <- "Jane"
last <- "Doe"

# Combine strings
str_c(first, last)           # No space
```

```
[1] "JaneDoe"
```

```
str_c(first, last, sep = " ")  # With space
```

```
[1] "Jane Doe"
```

```
# Combine with other text
str_c("Participant: ", first, " ", last)
```

```
[1] "Participant: Jane Doe"
```

```
# Glue is even easier (from the glue package)
library(glue)
glue("Participant: {first} {last}")
```

```
Participant: Jane Doe
```

String length

```
responses <- c("Yes", "No", "Maybe", "I don't know")

str_length(responses)
```

```
[1]  3  2  5 12
```

Useful for checking free-response data quality:

```
# Flag very short responses
tibble(responses) |>
  mutate(too_short = str_length(responses) < 3)
```

```
# A tibble: 4 × 2
  responses    too_short
  <chr>        <lgl>
1 Yes          FALSE
2 No           TRUE
3 Maybe        FALSE
4 I don't know FALSE
```

Changing case

```
messy_data <- c("PSYCHOLOGY", "Biology", "psychology", "BIOLOGY", "Psychology

str_to_lower(messy_data)
```

```
[1] "psychology" "biology"    "psychology" "biology"    "psychology"
```

```
str_to_upper(messy_data)
```

```
[1] "PSYCHOLOGY" "BIOLOGY"    "PSYCHOLOGY" "BIOLOGY"    "PSYCHOLOGY"
```

```
str_to_title(messy_data)
```

```
[1] "Psychology" "Biology"    "Psychology" "Biology"    "Psychology"
```

Essential for cleaning survey data!

Trimming whitespace

Survey data often has extra spaces:

```
messy_responses <- c("  Yes", "No  ", "  Maybe  ")

str_trim(messy_responses)         # Remove leading/trailing
```

```
[1] "Yes"   "No"    "Maybe"
```

```
str_squish(messy_responses)       # Also reduce internal spaces
```

```
[1] "Yes"   "No"    "Maybe"
```

Psychology example: Cleaning survey responses

```
survey <- tibble(
  major = c("  Psychology", "BIOLOGY  ", "psychology", "Biology", "  sociolog
)

survey |>
  mutate(
    major_clean = str_to_lower(str_trim(major))
  )
```

```
# A tibble: 5 × 2
  major          major_clean
  <chr>          <chr>
1 "  Psychology" psychology
2 "BIOLOGY  "    biology
3 "psychology"   psychology
4 "Biology"      biology
5 "  sociology"  sociology
```

Detecting patterns

`str_detect()` checks if a pattern is present:

```
feedback <- c(
  "The task was clear",
  "I found it confusing",
  "Very clear instructions",
  "Somewhat confusing"
)

str_detect(feedback, "clear")
```

```
[1]  TRUE FALSE  TRUE FALSE
```

Detecting patterns in `filter()`

`str_detect()` returns `TRUE`/`FALSE` — exactly what `filter()` needs:

```
tibble(feedback) |>
  filter(str_detect(feedback, "clear"))
```

```
# A tibble: 2 × 1
  feedback
  <chr>
1 The task was clear
2 Very clear instructions
```

Case-insensitive detection

Patterns are case-sensitive by default:

```
str_detect("Clear instructions", "clear")  # FALSE
```

```
[1] FALSE
```

Make them case-insensitive with `regex(ignore_case = TRUE)`:

```
str_detect("Clear instructions", regex("clear", ignore_case = TRUE))
```

```
[1] TRUE
```

Replacing text

Open-ended responses often have repeated words:

```
responses <- c(
  "very very tired",
  "very stressed and very anxious",
  "I felt fine"
)

# Replace the first match in each string
str_replace(responses, "very", "extremely")
```

```
[1] "extremely very tired"                "extremely stressed and very
anxious"
[3] "I felt fine"
```

```
# Replace every match in each string
str_replace_all(responses, "very", "extremely")
```

```
[1] "extremely extremely tired"
[2] "extremely stressed and extremely anxious"
[3] "I felt fine"
```

Multiple replacements

Imagine clinical notes with diagnosis abbreviations scattered inside the text:

```
notes <- c(
  "Patient presented with MDD symptoms",
  "Differential: MDD vs GAD",
  "Family history of OCD and GAD"
)
```

Multiple replacements

Pass a named vector to `str_replace_all()` to expand them **wherever they appear**:

```
str_replace_all(notes, c(
  "MDD" = "Major Depressive Disorder",
  "GAD" = "Generalized Anxiety Disorder",
  "OCD" = "Obsessive-Compulsive Disorder"
))
```

```
[1] "Patient presented with Major Depressive Disorder symptoms"
[2] "Differential: Major Depressive Disorder vs Generalized Anxiety Disorder"
[3] "Family history of Obsessive-Compulsive Disorder and Generalized Anxiety
Disorder"
```

Extracting parts of strings

```
participant_ids <- c("PSY001", "PSY002", "PSY010", "PSY123")

# Extract substring by position
str_sub(participant_ids, start = 4)  # Get everything after position 3
```

```
[1] "001" "002" "010" "123"
```

```
# Extract numbers
str_extract(participant_ids, "\\d+")  # \d+ means "one or more digits"
```

```
[1] "001" "002" "010" "123"
```

When you need more: Regular expressions

Regular expressions (regex) are powerful pattern matching tools.

Examples:

- `\\d` matches digits
- `\\s` matches whitespace
- `.` matches any character
- `+` means “one or more”
- `*` means “zero or more”

> **Note**
>
> Regex is powerful but complex. For this course, stick to simple patterns. When you need more, check R4DS Ch 14 or regex101.com.

Pair coding break

Your turn: Clean text data

You have messy survey responses:

```
messy_survey <- tibble(
  id = 1:5,
  major = c("  PSYCHOLOGY", "biology  ", "Psychology", "BIOLOGY", "sociology
  comment = c(
    "Great study!",
    "too long",
    "  Very interesting  ",
    "CONFUSING INSTRUCTIONS",
    "I enjoyed this"
  )
)
```

Your turn: Clean text data

1. Clean `major` to lowercase with no extra spaces
2. Clean `comment` to title case with no extra spaces
3. Create a logical column `is_negative` that is `TRUE` if the comment contains "long" or "confusing" (case-insensitive)
4. Filter to only negative comments

Time: 10 minutes

Before we move on

📤 Submit your code on Canvas for participation credit. Paste what you have — it doesn't need to work perfectly.

Factors

What are factors?

Factors are R's way of representing categorical data with a fixed set of possible values.

```
# A character vector
major_char <- c("Psychology", "Biology", "Biology", "Psychology")

# A factor
major_fct <- factor(major_char)
major_fct
```

```
[1] Psychology Biology    Biology    Psychology
Levels: Biology Psychology
```

Notice the **Levels** line — those are the possible categories.

Why use factors?

1. **Memory efficient** — R stores categories once, not repeatedly
2. **Prevent typos** — Can't accidentally add invalid categories
3. **Control order** — Specify the order for plots and tables
4. **Model requirements** — Many statistical models require factors

Creating factors

```r
condition <- c("Control", "Treatment", "Treatment", "Control")

# Let R choose levels (alphabetical)
factor(condition)
```

```
[1] Control   Treatment Treatment Control
Levels: Control Treatment
```

```r
# Specify levels explicitly
factor(condition, levels = c("Control", "Treatment"))
```

```
[1] Control   Treatment Treatment Control
Levels: Control Treatment
```

Order matters!

```
likert <- c("Agree", "Disagree", "Strongly Agree", "Agree", "Strongly Disagre

# Alphabetical order (default)
factor(likert)
```

```
[1] Agree             Disagree          Strongly Agree    Agree
[5] Strongly Disagree
Levels: Agree Disagree Strongly Agree Strongly Disagree
```

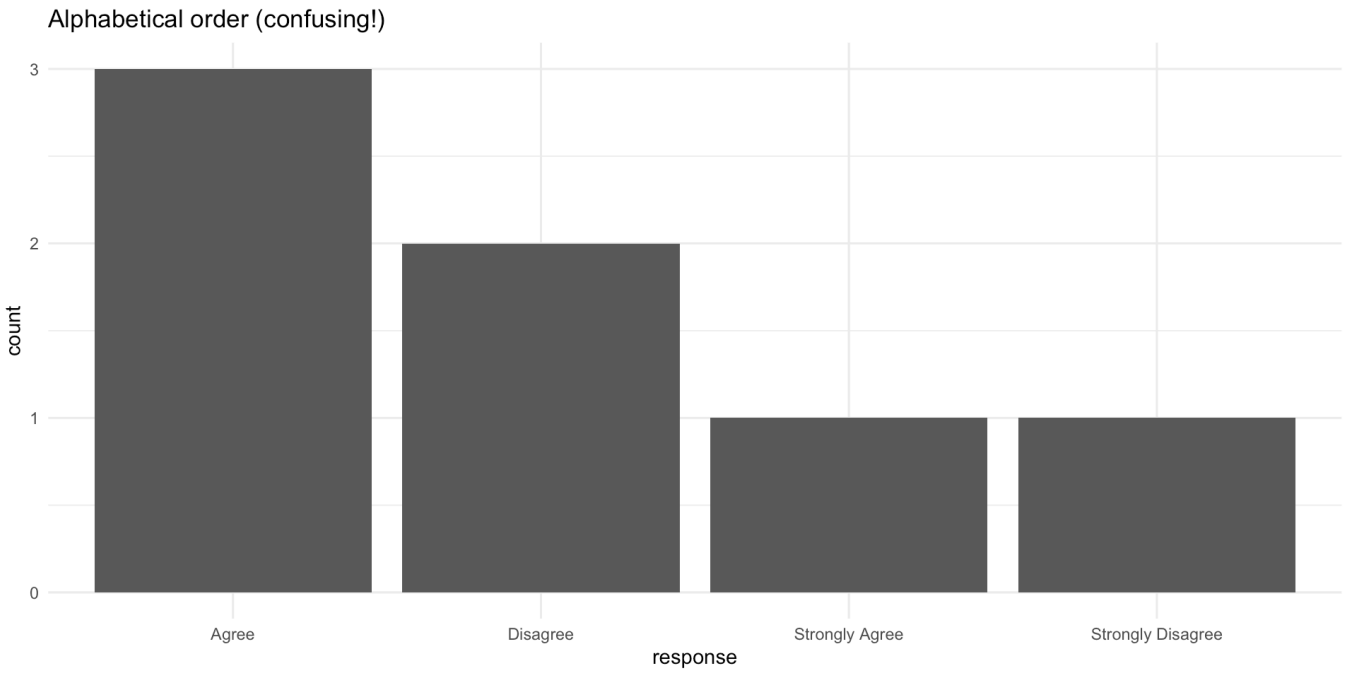
```
# Logical order
factor(likert, levels = c(
  "Strongly Disagree", "Disagree", "Agree", "Strongly Agree"
))
```

```
[1] Agree             Disagree          Strongly Agree    Agree
[5] Strongly Disagree
Levels: Strongly Disagree Disagree Agree Strongly Agree
```

Why order matters: Example

```
survey_data <- tibble(
  response = c("Agree", "Disagree", "Strongly Agree", "Agree",
               "Strongly Disagree", "Agree", "Disagree")
)

# Without factor ordering
ggplot(survey_data, aes(x = response)) +
  geom_bar() +
  labs(title = "Alphabetical order (confusing!)") +
  theme_minimal()
```

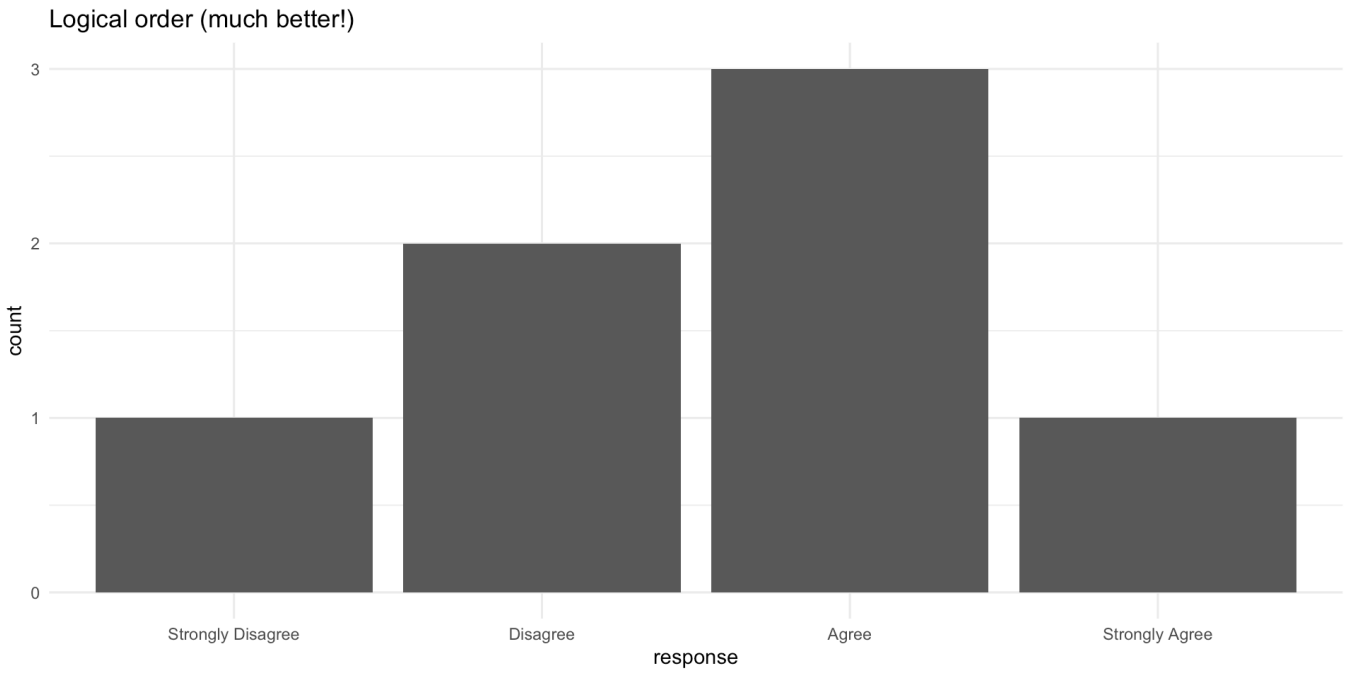
Why order matters: Example



Fixed with factor ordering

```
survey_data <- survey_data |>
  mutate(
    response = factor(response, levels = c(
      "Strongly Disagree", "Disagree", "Agree", "Strongly Agree"
    ))
  )

ggplot(survey_data, aes(x = response)) +
  geom_bar() +
  labs(title = "Logical order (much better!)") +
  theme_minimal()
```

Fixed with factor ordering



Forcats: Factor tools

The `forcats` package (part of tidyverse) provides functions for working with factors.

All `forcats` functions start with `fct_`.

Key functions:

- `fct_relevel()` — manually reorder levels
- `fct_reorder()` — reorder by another variable
- `fct_infreq()` — order by frequency
- `fct_recode()` — rename levels
- `fct_collapse()` — combine levels

fct_relevel(): Manual reordering

```
education <- factor(c("High School", "Bachelor's", "Master's", "High School"))
education
```

```
[1] High School Bachelor's  Master's    High School
Levels: Bachelor's High School Master's
```

```
# Put in logical order
fct_relevel(education, "High School", "Bachelor's", "Master's")
```

```
[1] High School Bachelor's  Master's    High School
Levels: High School Bachelor's Master's
```

fct_reorder(): Order by another variable

Extremely useful for plots!

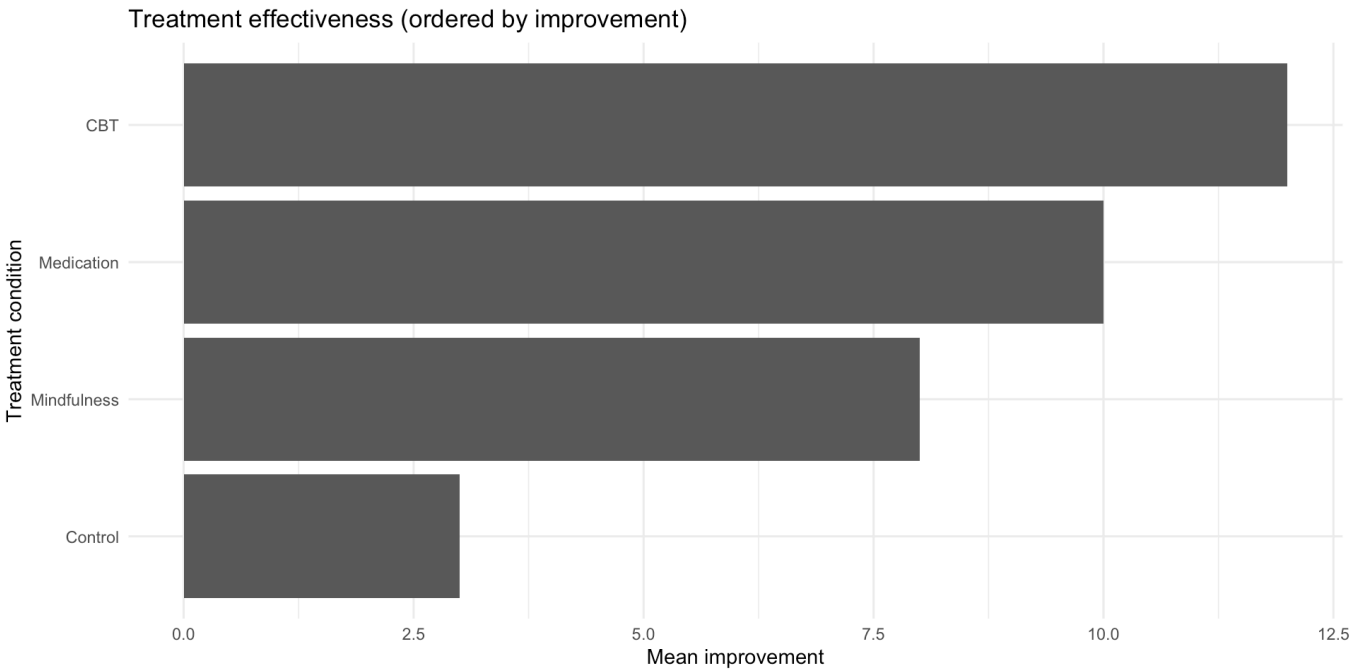
```
therapy <- tibble(
  condition = c("CBT", "Control", "Mindfulness", "Medication"),
  mean_improvement = c(12, 3, 8, 10)
)

therapy |>
  mutate(condition = fct_reorder(condition, mean_improvement))
```

```
# A tibble: 4 × 2
  condition   mean_improvement
  <fct>                  <dbl>
1 CBT                       12
2 Control                    3
3 Mindfulness                8
4 Medication                10
```

fct_reorder() in action

```
therapy |>
  mutate(condition = fct_reorder(condition, mean_improvement)) |>
  ggplot(aes(x = mean_improvement, y = condition)) +
  geom_col() +
  labs(
    title = "Treatment effectiveness (ordered by improvement)",
    x = "Mean improvement",
    y = "Treatment condition"
  ) +
  theme_minimal()
```

fct_reorder() in action



fct_infreq(): Order by frequency

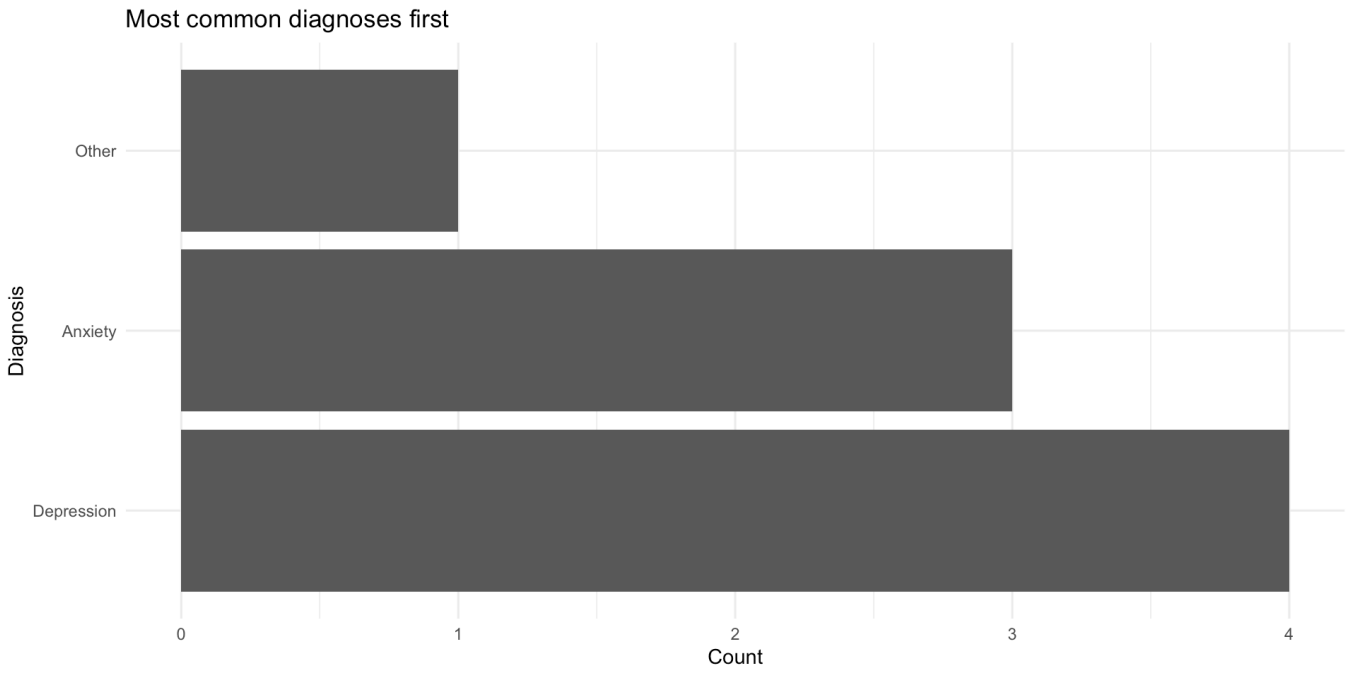
```
diagnosis <- c("Depression", "Anxiety", "Depression", "Other",
               "Depression", "Anxiety", "Anxiety", "Depression")

# Order from most to least common
fct_infreq(factor(diagnosis))
```

```
[1] Depression Anxiety    Depression Other      Depression Anxiety    Anxiety
[8] Depression
Levels: Depression Anxiety Other
```

fct_infreq() in plots

```
tibble(diagnosis = factor(diagnosis)) |>
  mutate(diagnosis = fct_infreq(diagnosis)) |>
  ggplot(aes(y = diagnosis)) +  # Note: y instead of x to read easily
  geom_bar() +
  labs(
    title = "Most common diagnoses first",
    x = "Count",
    y = "Diagnosis"
  ) +
  theme_minimal()
```

fct_infreq() in plots



fct_recode(): Rename levels

```
major <- factor(c("Psych", "Bio", "Bio", "Psych", "Soc"))

fct_recode(major,
  "Psychology" = "Psych",
  "Biology" = "Bio",
  "Sociology" = "Soc"
)
```

```
[1] Psychology Biology    Biology    Psychology Sociology
Levels: Biology Psychology Sociology
```

fct_collapse(): Combine levels

Useful for grouping rare categories:

```
diagnosis <- factor(c("MDD", "GAD", "OCD", "PTSD", "Panic Disorder",
                      "MDD", "GAD", "Social Anxiety"))

fct_collapse(diagnosis,
  Depression = "MDD",
  Anxiety = c("GAD", "OCD", "PTSD", "Panic Disorder", "Social Anxiety")
)
```

```
[1] Depression Anxiety    Anxiety    Anxiety    Anxiety    Depression Anxiety
[8] Anxiety
Levels: Anxiety Depression
```

Psychology example: Recoding demographics

```
demo_data <- tibble(
  age_group = c("18–25", "26–35", "18–25", "36–45", "26–35", "46+"),
  education = c("HS", "BA", "BA", "MA", "HS", "PhD")
)

demo_data
```

```
# A tibble: 6 × 2
  age_group education
  <chr>     <chr>
1 18–25     HS
2 26–35     BA
3 18–25     BA
4 36–45     MA
5 26–35     HS
6 46+       PhD
```

Recoding demographics

```
demo_data |>
  mutate(
    age_group = factor(age_group, levels = c("18–25", "26–35", "36–45", "46+"
    education = fct_recode(factor(education),
      "High School" = "HS",
      "Bachelor's"  = "BA",
      "Master's"    = "MA",
      "Doctorate"   = "PhD"
    )
  )
```

```
# A tibble: 6 × 2
  age_group education
  <fct>     <fct>
1 18–25     High School
2 26–35     Bachelor's
3 18–25     Bachelor's
4 36–45     Master's
5 26–35     High School
6 46+       Doctorate
```

Dropping unused levels

After filtering, factors keep old levels:

```
all_diagnoses <- factor(c("Depression", "Anxiety", "Other"))
just_depression <- all_diagnoses[all_diagnoses == "Depression"]
just_depression
```

```
[1] Depression
Levels: Anxiety Depression Other
```

Use `fct_drop()` to remove them:

```
fct_drop(just_depression)
```

```
[1] Depression
Levels: Depression
```

Common factor issues

Problem 1: Factors created from numbers

```
age_factor <- factor(c(25, 30, 25, 40))
mean(age_factor)  # Error! It's not numeric anymore
```

```
[1] NA
```

Solution: Convert back to numeric carefully:

```
as.numeric(as.character(age_factor))  # Correct way
```

```
[1] 25 30 25 40
```

Common factor issues

Problem 2: Factors behave differently than strings

```
colors <- factor(c("red", "blue"))

# Can't just add new values
colors[3] <- "green"  # This creates NA!
colors
```

```
[1] red  blue <NA>
Levels: blue red
```

Solution: Convert to character first, or use `fct_expand()` to add levels.

Light text analysis

What did you say about data?

On the class survey, you described your relationship with data in 1–2 words. Let's see what you said.

```
class_survey <- read_csv(
  "https://raw.githubusercontent.com/sjweston/datasci410/refs/heads/main/data
)
```

```
class_survey |>
  select(data_words) |>
  head(10)
```

```
# A tibble: 10 × 1
   data_words
   <chr>
 1 frustrating and overwhelming
 2 novel
 3 novice
 4 hopeful
 5 work-in-progress
 6 big and scary
 7 necessary
 8 theory adjacent
 9 unrequited love
10 neutral
```

Tokenizing text with tidytext

`unnest_tokens()` splits text into individual words:

```
words <- class_survey |>
  select(data_words) |>
  unnest_tokens(word, data_words)

words
```

```
# A tibble: 77 × 1
   word
   <chr>
 1 frustrating
 2 and
 3 overwhelming
 4 novel
 5 novice
 6 hopeful
 7 work
 8 in
 9 progress
10 big
# ℹ 67 more rows
```

Each row is now a single word.

Words alone aren't enough

```
words |>
  anti_join(stop_words, by = "word") |>
  count(word, sort = TRUE) |>
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
  word            n
  <chr>       <int>
1 complicated     2
2 curious         2
3 love            2
4 adjacent        1
5 barely          1
```

Most words appeared just once. Counting alone won't tell us much.

But every word carries a **feeling** — positive, negative, neutral. Can we measure *that*?

How does our class feel about data?

`tidytext` ships with the **Bing lexicon** — ~6,800 English words tagged `positive` or `negative`. We can `inner_join()` to keep only the words our class used *and* the lexicon recognized.

```
words |>
  inner_join(get_sentiments("bing"), by = "word") |>
  count(sentiment) |>
  ggplot(aes(x = sentiment, y = n, fill = sentiment)) +
  geom_col() +
  scale_fill_manual(values = c("negative" = "#c0392b", "positive" = "#27ae60"
  labs(
    title = "How our class feels about data",
    x = NULL, y = "Words"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "none")
```

How does our class feel about data?

Which words drove the score?

```
words |>
  inner_join(get_sentiments("bing"), by = "word") |>
  count(word, sentiment, sort = TRUE)
```

```
# A tibble: 26 × 3
   word        sentiment     n
   <chr>       <chr>     <int>
 1 good        positive      3
 2 complicated negative      2
 3 interesting positive      2
 4 love        positive      2
 5 brutal      negative      1
 6 confused    negative      1
 7 derogatory  negative      1
 8 difficult   negative      1
 9 frustrating negative      1
10 fun         positive      1
# ℹ 16 more rows
```

Read carefully — did the lexicon get every word **right**?

Where did “good” actually come from?

`unnest_tokens(..., drop = FALSE)` keeps the source phrase so we can spot-check:

```
class_survey |>
  select(data_words) |>
  unnest_tokens(word, data_words, drop = FALSE) |>
  inner_join(get_sentiments("bing"), by = "word") |>
  filter(word %in% c("good", "great", "love")) |>
  mutate(data_words = str_trunc(data_words, 35))
```

```
# A tibble: 6 × 3
  data_words                          word  sentiment
  <chr>                               <chr> <chr>
1 unrequited love                     love  positive
2 not great                           great positive
3 not good                            good  positive
4 i have never coded before and it... good  positive
5 good                                good  positive
6 love-hate                           love  positive
```

End-of-deck exercise

Practice: Clean and visualize survey data

You have messy Likert scale data:

```
likert_data <- tibble(
  question = rep(c("Q1", "Q2", "Q3"), each = 10),
  response = sample(c("strongly agree", "Agree", "NEUTRAL",
                      "disagree", "Strongly Disagree"), 30, replace = TRUE)
)
```

1. Clean the `response` variable to title case
2. Convert `response` to a factor with logical ordering
3. Create a bar chart showing response counts for each question
4. Use `facet_wrap()` to make separate panels for each question
5. Bonus: Use `fct_infreq()` to order responses by overall frequency
6. **A6 preview** — given an `open_responses` tibble with a `response` column, tokenize it and find the most common non-stopwords:

```
library(tidytext)
open_responses |>
  unnest_tokens(word, response) |>
  anti_join(stop_words, by = "word") |>
  count(word, sort = TRUE)
```

Wrapping up

String functions cheat sheet

Function	Purpose
`str_c()`	Combine strings
`str_length()`	Get string length
`str_to_lower()`, `str_to_upper()`	Change case
`str_trim()`, `str_squish()`	Remove whitespace
`str_detect()`	Find pattern
`str_replace()`, `str_replace_all()`	Replace pattern
`str_sub()`	Extract substring

Factor functions cheat sheet

Function	Purpose
`factor()`	Create a factor
`fct_relevel()`	Manually reorder levels
`fct_reorder()`	Order by another variable
`fct_infreq()`	Order by frequency
`fct_recode()`	Rename levels
`fct_collapse()`	Combine levels
`fct_drop()`	Remove unused levels

Key takeaways

1. **Strings** are text — use `stringr::str_*()` functions to manipulate
2. **Always clean string data** — case, whitespace, typos
3. **Factors** are categorical data with fixed levels
4. **Factor order matters** for plots and tables
5. **Use forcats** (`fct_*()`) to manipulate factors
6. **Order factors logically** — not alphabetically
7. When in doubt, **check the data type** with `class()` or `glimpse()`

Before next class

📖 **Read:**

- R4DS Ch 18: Missing values

✅ **Do:**

- Start Assignment 6
- Check your final project data for string/factor issues
- Practice cleaning demographic variables

The one thing to remember

Messy categories turn into messy results. `str_to_lower()` and `factor()` are your first line of defense.

See you Wednesday for missing data!